

? Conditional (Ternary) Operator in C

Definition

The **conditional operator** (also called the **ternary operator**) is a shortcut for writing a simple **if...else** statement in a single line. It is called "ternary" because it is the **only operator in C that works on three operands** (a condition, a value if true, and a value if false).

Syntax

```
condition ? expression1 : expression2;
```

- condition** → an expression that is evaluated first. It results in **true (non-zero)** or **false (0)**.
- If the condition is **true**, **expression1** is evaluated and becomes the result.
- If the condition is **false**, **expression2** is evaluated and becomes the result.

How It Works

Think of it as asking a question: *"Is this condition true? If yes, do this — otherwise, do that."*

Statement using if...else	Same statement using ternary operator
<pre>if (a > b) max = a; else max = b;</pre>	<pre>max = (a > b) ? a : b;</pre>

Both do exactly the same job — the ternary operator just writes it in **one clean line**.

Example Program

```
// Program to find the greatest of two numbers using ternary operator
#include <stdio.h>

int main() {
    int a = 10, b = 20, max;

    max = (a > b) ? a : b;

    printf("The greatest number is: %d\n", max);

    return 0;
}
```

Output

```
The greatest number is: 20
```

Note

- The ternary operator can also be **nested** (one ternary operator inside another) to check multiple conditions, but this should be used carefully — too much nesting is hard to read.
- The ternary operator is often used to make code **shorter**, but for complex conditions, a normal **if...else** is easier to understand.

Example: Nested Ternary Operator

```
// Program to check if a number is positive, negative, or zero
#include <stdio.h>

int main() {
    int num = 0;

    (num == 0) ? printf("Zero") :
    (num > 0) ? printf("Positive") : printf("Negative");

    return 0;
}
```

Output

```
Zero
```

Fun Tip

? : → think of it as a mini **if-else** that fits in one line!

Operator Types in C (Unary, Binary, Ternary)

Definition

Operators in C can be grouped by **how many operands (values) they work on**. This is called the "**arity**" of an operator. Based on arity, C operators are divided into **three types**: Unary, Binary, and Ternary.

How It Works

Type	No. of Operands	Meaning	Example Operators	Example
Unary	1	Works on a single operand	++, --, - (unary minus), !, &, sizeof	-a, ++x, !flag
Binary	2	Works on two operands	+, -, *, /, &&, ==, =	a + b, x == y
Ternary	3	Works on three operands	? : (conditional operator)	a > b ? a : b

Think of it like this: "Unary" = uni (one), "Binary" = bi (two), "Ternary" = ter (three) — the name itself tells you how many operands are needed!

Example Program

```
#include <stdio.h>

int main() {
    int a = 5, b = 3, result;

    // Unary operator: works on ONE operand (a)
    result = -a;
    printf("Unary (-a)      : %d\n", result);

    // Binary operator: works on TWO operands (a, b)
    result = a + b;
    printf("Binary (a + b)   : %d\n", result);

    // Ternary operator: works on THREE operands
    result = (a > b) ? a : b;
    printf("Ternary (a>b?a:b): %d\n", result);

    return 0;
}
```

Output

```
Unary (-a)      : -5
Binary (a + b)   : 8
Ternary (a>b?a:b): 5
```

Note

- C has **only one** ternary operator: ? :. There is no way to create a custom ternary operator.
- Most operators you will use daily (arithmetic, relational, logical, assignment) are **binary**.
- ++ and -- are unary but special — they can appear **before** (pre) or **after** (post) the variable.

 **Fun Tip**

1 operand = **Unary** | 2 operands = **Binary** | 3 operands = **Ternary**

📖 Definition

An **expression** is any valid combination of **operators, constants, and variables** that is evaluated to produce a **single value**.
In simple words — anything in your C program that can be **calculated to give a result** is an expression.

🔗 How It Works

Expression	Type	Result
5	Constant expression	5
a	Variable expression	value stored in <code>a</code>
a + b	Arithmetic expression	sum of a and b
a > b	Relational expression	1 (true) or 0 (false)
(a > b) && (c > d)	Logical expression	1 or 0
a = b + c	Assignment expression	value assigned to <code>a</code>

🔗 Types of Expressions

- **Constant Expression:** made up of only constant values. Example: 5 + 10
- **Integer Expression:** an expression that produces an integer result. Example: a * b (when a, b are int)
- **Floating-point Expression:** produces a decimal (float/double) result. Example: a / 2.0
- **Relational Expression:** compares two values, gives true/false. Example: a <= b
- **Logical Expression:** combines relational expressions. Example: a > 0 && b > 0
- **Bitwise Expression:** works on bits of data. Example: a & b

📄 Example Program

```
#include <stdio.h>

int main() {
    int a = 8, b = 3;

    printf("Arithmetic expression a+b : %d\n", a + b);
    printf("Relational expression a>b : %d\n", a > b);
    printf("Logical expression a>0&&b>0 : %d\n", a > 0 && b > 0);
    printf("Float expression a/2.0 : %.2f\n", a / 2.0);

    return 0;
}
```

▶ Output

```
Arithmetic expression a+b : 11
Relational expression a>b : 1
Logical expression a>0&&b>0 : 1
Float expression a/2.0 : 4.00
```

📌 Note

- Every expression, when evaluated, produces **exactly one value**.
- A **statement** may contain one or more expressions, followed by a semicolon ;
- Example: a = b + c; is a **statement**, while b + c is the **expression** inside it.

🔗 Fun Tip

If it can be **calculated** to give **one value** → it's an **expression**!

Order of Expression (Precedence & Associativity)

Definition

When an expression has **more than one operator**, C needs rules to decide **which operator to evaluate first**. These rules are called:

- **Precedence** → decides **which operator** is evaluated first when different operators appear together.
- **Associativity** → decides the **direction** (left-to-right or right-to-left) in which operators of the **same precedence** are evaluated.

How It Works

Example: $10 + 5 * 2$

- $*$ has **higher precedence** than $+$, so multiplication happens first.
- Step 1: $5 * 2 = 10$
- Step 2: $10 + 10 = 20$

So the result is **20**, not 30 — because $*$ is evaluated before $+$.

Precedence Table (High to Low, common operators)

Precedence	Operator	Description	Associativity
1 (Highest)	<code>() []</code>	Parentheses, Array subscript	Left to Right
2	<code>++ -- ! - (unary) sizeof</code>	Unary operators	Right to Left
3	<code>* / %</code>	Multiplication, Division, Modulus	Left to Right
4	<code>+ -</code>	Addition, Subtraction	Left to Right
5	<code>< <= > >=</code>	Relational operators	Left to Right
6	<code>== !=</code>	Equality operators	Left to Right
7	<code>&&</code>	Logical AND	Left to Right
8	<code> </code>	Logical OR	Left to Right
9	<code>? :</code>	Conditional (ternary)	Right to Left
10 (Lowest)	<code>= += -= *= ...</code>	Assignment operators	Right to Left

☞ You don't need to memorize the entire table — just remember: **parentheses first**, then **unary**, then **arithmetic ($*$ / $%$ before $+$ -), then **comparisons**, then **logical**, and **assignment last**.**

Example Program

```
#include <stdio.h>

int main() {
    int result1 = 10 + 5 * 2;    // * before +
    int result2 = (10 + 5) * 2;  // () changes order
    int result3 = 20 / 5 % 3;    // / and % : left to right

    printf("10 + 5 * 2    = %d\n", result1);
    printf("(10 + 5) * 2 = %d\n", result2);
    printf("20 / 5 % 3    = %d\n", result3);

    return 0;
}
```

Output

```
10 + 5 * 2    = 20
(10 + 5) * 2 = 30
20 / 5 % 3    = 1
```

Step-by-Step: How $20 / 5 \% 3$ is evaluated

1. $/$ and $\%$ have the **same precedence**, so associativity decides the order → **Left to Right**.
2. Step 1: $20 / 5 \rightarrow 4$ (leftmost operator evaluated first)
3. Step 2: $4 \% 3 \rightarrow 1$
4. Final result → **1**

Note

- When in doubt, always use **parentheses ()** to make the order clear — it also makes your code easier to read.
- **Precedence** decides "which operator first"; **Associativity** decides "which direction" when precedence is equal.
- Assignment operators ($=$, $+=$, etc.) always execute **last** and go **right to left**.

 **Fun Tip**

PEMDAS from maths class = same idea as **Precedence** in C! 💡